VIT-AP UNIVERSITY

Semester: FALL 2026-27

Course Name: Software Application Architecture

Course Code: SWE4007

PROJECT TITLE :

SOCIAL MEDIA PLATFORM WITH REAL-TIME CHATTING

Team Members:

1.	D BinduSri	23MIC7266
2.	B Tharuni Sri Sai	23MIC7268
3.	G Akhil	23MIC7269
4.	M Ramya Sree	23MIC7275
5.	P Yaswanth	23MIC7281

Supervisor :

Under the Guidance of

Dr. Tauseef Khan

Department of Computer Science and Engineering

VIT-AP University

Amaravati, Andhra Pradesh

1. INTRODUCTION

1.1 Purpose

The Social Media Platform with Real-Time Chatting is a web-based application that integrates social networking and real-time communication into a single platform. It enables users to create profiles, connect with other users, share and interact with posts, discover content, and communicate through one-to-one and group conversations.

The system is designed using a microservices architecture, with independent services for authentication, users, posts, chat, notifications, media, moderation, and administration. REST APIs are used for standard application operations, while WebSocket-based communication enables real-time messaging, typing indicators, online presence, delivery status, and read receipts. Asynchronous events are handled through a message broker to provide loosely coupled service communication.

The SRS defines the functional and non-functional requirements, network configuration, system architecture, and UML models required for the development of the proposed platform.

1.2 Scope

The system provides the following major capabilities:

- Authentication and Profile Management: registration, login, secure authentication, profile creation, and profile updates.
- Social Networking: user search, follow/unfollow, followers/following, blocking, and muting.
- Content Management: creation and management of text/image posts, personalized feed, likes, reactions, comments, replies, shares, bookmarks, hashtags, and mentions.
- Real-Time Communication: one-to-one and group chats, instant messaging, attachments, typing indicators, online/offline status, delivery status, read receipts, and message reactions.
- Notifications: real-time notifications for messages, follows, likes, comments, and mentions.
- Privacy and Safety: account privacy, communication controls, blocking, muting, and reporting.
- Moderation and Administration: report management, content moderation, user management, and platform statistics.

The system is intended to provide a secure, scalable, maintainable, and responsive platform with a modern user experience.

1.3 Intended Users

The primary users are:

General Users: Users who create profiles, connect with others, share content, interact with posts, and communicate through real-time chat.

Administrators: Authorized users responsible for managing accounts, reviewing reports, moderating content, and monitoring platform activities.

2. OBJECTIVES

The primary objective of the system is to develop a secure, scalable, and user-friendly social media platform that combines content sharing, social networking, and real-time communication within a unified application.

The specific objectives are to:

1. Provide secure authentication and authorization for protecting user accounts and resources.
2. Enable users to create profiles, build social connections, and discover other users and content.
3. Provide efficient post creation, sharing, interaction, and personalized feed management.
4. Deliver low-latency real-time communication through one-to-one and group conversations.
5. Support real-time features including typing indicators, presence, delivery status, and read receipts.
6. Provide real-time notifications for important social and communication events.
7. Ensure privacy and user safety through blocking, muting, reporting, and configurable privacy controls.
8. Provide administrators with moderation and user-management capabilities.
9. Use a microservices architecture to improve modularity, scalability, maintainability, and fault isolation.
10. Provide a responsive and intuitive user interface across different screen sizes.

3. FUNCTIONAL REQUIREMENTS

The system shall provide the following functional capabilities:

3.1 Authentication and Account Management

- **FR-01:** The system shall allow users to register, log in, log out, and securely manage their accounts.
- **FR-02:** The system shall provide authentication and authorization using secure tokens and role-based access control.
- **FR-03:** Users shall be able to change/reset passwords and manage account settings.

3.2 Profile and User Management

- **FR-04:** Users shall be able to create, view, and update their profiles, including name, username, bio, profile picture, and cover image.
- **FR-05:** Users shall be able to search for and view other user profiles.
- **FR-06:** Users shall be able to follow/unfollow users and manage followers and following lists.
- **FR-07:**: Users shall be able to block or mute other users.

3.3 Posts and Feed

- **FR-08:** Users shall be able to create, edit, delete, and view text/image posts.
- **FR-09:** Users shall be able to specify post visibility and use hashtags and mentions.
- **FR-10:** The system shall provide a personalized feed containing relevant posts.
- **FR-11:** Users shall be able to like/react to, comment on, reply to, share, and bookmark posts.

3.4 Search and Content Discovery

- **FR-12:** The system shall support searching for users, posts, and hashtags.
- **FR-13:** The system shall provide content discovery, trending content, and suggested users.

3.5 Real-Time Chat

- **FR-14:** Users shall be able to initiate one-to-one conversations and create group conversations.

- **FR-15:** Users shall be able to send and receive text messages and supported media/files in real time.
- **FR-16:** The system shall maintain conversation and message history.
- **FR-17:** Users shall be able to reply to, react to, edit, and delete messages where permitted.

3.6 Real-Time Communication Features

- **FR-18:** The system shall provide real-time typing indicators and online/offline presence.
- **FR-19:** The system shall provide message sent, delivered, and read statuses.
- **FR-20:** The system shall maintain unread message counts and support reconnection after temporary network interruption.

3.7 Notifications

- **FR-21:** The system shall generate notifications for follows, likes, comments, mentions, messages, and other relevant activities.
- **FR-22:** Notifications shall be updated in real time and users shall be able to mark them as read.

3.8 Media Management

- **FR-23:** Users shall be able to upload and manage supported images and files for profiles, posts, and conversations.
- **FR-24:** The system shall validate uploaded files based on supported type and size.

3.9 Privacy and Safety

- **FR-25:** Users shall be able to configure account privacy and messaging preferences.
- **FR-26:** Users shall be able to report users, posts, comments, and messages.
- **FR-27:** Users shall be able to block or mute unwanted interactions.

3.10 Moderation and Administration

- **FR-28:** Administrators shall be able to view and manage registered users.
- **FR-29:** Administrators shall be able to review reports and moderate reported content.
- **FR-30:** Authorized administrators shall be able to suspend, restrict, or restore user accounts.
- **FR-31:** The system shall provide administrative statistics and activity information.

3.11 System and Service Management

- **FR-32:** The system shall provide health monitoring and error handling for its distributed services.

- **FR-33:** Services shall communicate through defined APIs and asynchronous events where appropriate.
- **FR-34:** The system shall maintain appropriate logs and service activity information for troubleshooting and monitoring.

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance

- **NFR-01:** The system shall provide responsive user interactions and low-latency real-time message delivery under normal operating conditions.
- **NFR-02:** API requests and database operations shall be optimized to support concurrent users.

4.2 Scalability

- **NFR-03:** Individual microservices shall be independently scalable based on workload.
- **NFR-04:** The architecture shall support horizontal scaling of high-demand services such as Chat and Notification Services.

4.3 Security

- **NFR-05:** User credentials shall be securely stored using password hashing.
- **NFR-06:** The system shall implement authentication, authorization, input validation, rate limiting, and secure communication.
- **NFR-07:** Users shall only be permitted to access resources and operations for which they are authorized.

4.4 Reliability and Availability

- **NFR-08:** The system shall handle service or network failures gracefully without compromising stored data.
- **NFR-09:** Temporary connection failures in real-time communication shall support reconnection mechanisms.
- **NFR-10:** Critical services shall provide health checks and appropriate error handling.

4.5 Usability

- **NFR-11:** The user interface shall be intuitive, responsive, and accessible across desktop, tablet, and mobile screen sizes.
- **NFR-12:** The system shall provide clear feedback for successful operations, errors, loading states, and empty states.

4.6 Maintainability

- **NFR-13:** The system shall follow modular microservices design principles with clearly defined service responsibilities and interfaces.
- **NFR-14:** APIs and major system components shall be documented to simplify maintenance and future development.

4.7 Data Integrity and Consistency

- **NFR-15:** The system shall maintain data integrity through validation, appropriate database constraints, and controlled service-level access.
- **NFR-16:** Each microservice shall maintain ownership of its respective data.

4.8 Compatibility and Portability

- **NFR-17:** The application shall support modern web browsers and responsive web environments.
- **NFR-18:** Services shall be containerizable to support consistent development and deployment environments.

4.9 Observability

- **NFR-19:** The system shall provide centralized logging, service health information, and performance monitoring.
- **NFR-20:** Distributed service interactions shall be traceable to assist in identifying failures and performance issues.

5. NETWORK CONFIGURATION

The proposed system uses a distributed network configuration to support communication between the client, API Gateway, microservices, databases, and real-time communication infrastructure. All external communication is secured using HTTPS, while real-time communication uses WebSocket Secure (WSS). Internal services communicate through REST APIs and asynchronous events using a message broker.

5.1 Network Components

The network consists of the following major components:

- **Client:** Web browser running the Next.js frontend.
- **API Gateway:** Single entry point for client REST requests and routing to appropriate microservices.

- **Microservices**: Authentication, User, Post, Chat, Notification, Media, Moderation, and Administration services.
- **Real-Time Communication**: Chat Service uses WebSocket/WSS for instant message delivery and presence updates.
- **Message Broker:** RabbitMQ enables asynchronous communication between services through events.
- **Redis:** Provides caching, real-time presence, Pub/Sub, and coordination for real-time services.
- **Databases:** PostgreSQL and MongoDB provide persistent service-specific data storage.
- **Object Storage:** Stores profile images, post media, and chat attachments.

5.2 Communication Protocols

Communication	Protocol	Purpose
Client → API Gateway	HTTPS/REST	Application requests
Client → Chat Service	WSS/WebSocket	Real-time messaging
Gateway → Microservices	HTTP/REST	Synchronous service communication
Service → Service	REST	Immediate service interactions
Service → Message Broker	AMQP	Asynchronous events
Chat Services → Redis	Redis Protocol	Presence/Pub/Sub/cache
Services → Databases	Database Protocol	Data persistence
Media → Object Storage	HTTPS	File/media storage

5.3 Network Security

All client-facing communication shall use HTTPS/WSS to protect data during transmission. Authentication tokens shall be validated before accessing protected resources. API access shall be controlled through authentication, authorization, input validation, rate limiting, and appropriate security policies. Internal databases and infrastructure services shall not be directly exposed to external clients.